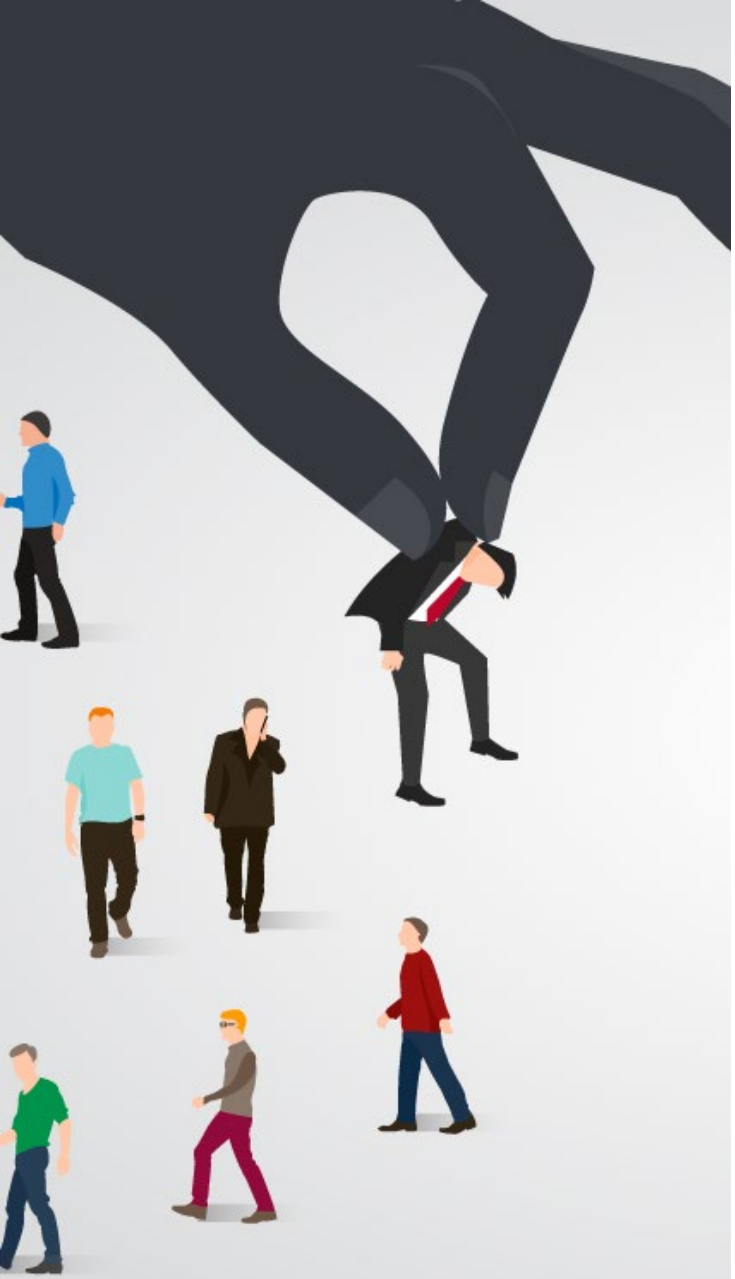
Model Selection





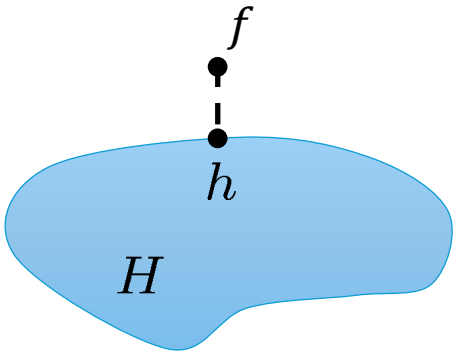
Topics

- Introduction
- **Overfitting and other Practical Issues**
- Model Selection and Evaluation
  - —Metrics for Performance Evaluation
  - —Methods to Obtain Reliable Estimates
  - —Model Comparison (Relative Performance)
- Feature Selection

Supervised Learning – Learning from Examples

- Examples
  - —Input-output pairs: $\mathrm{E} = (x_1, y_1), \ldots, (x_i, y_i), \ldots, (x_N, y_N)$.
  - —We assume that the examples are produced iid (with noise and errors) from a target function $y = f(x)$.

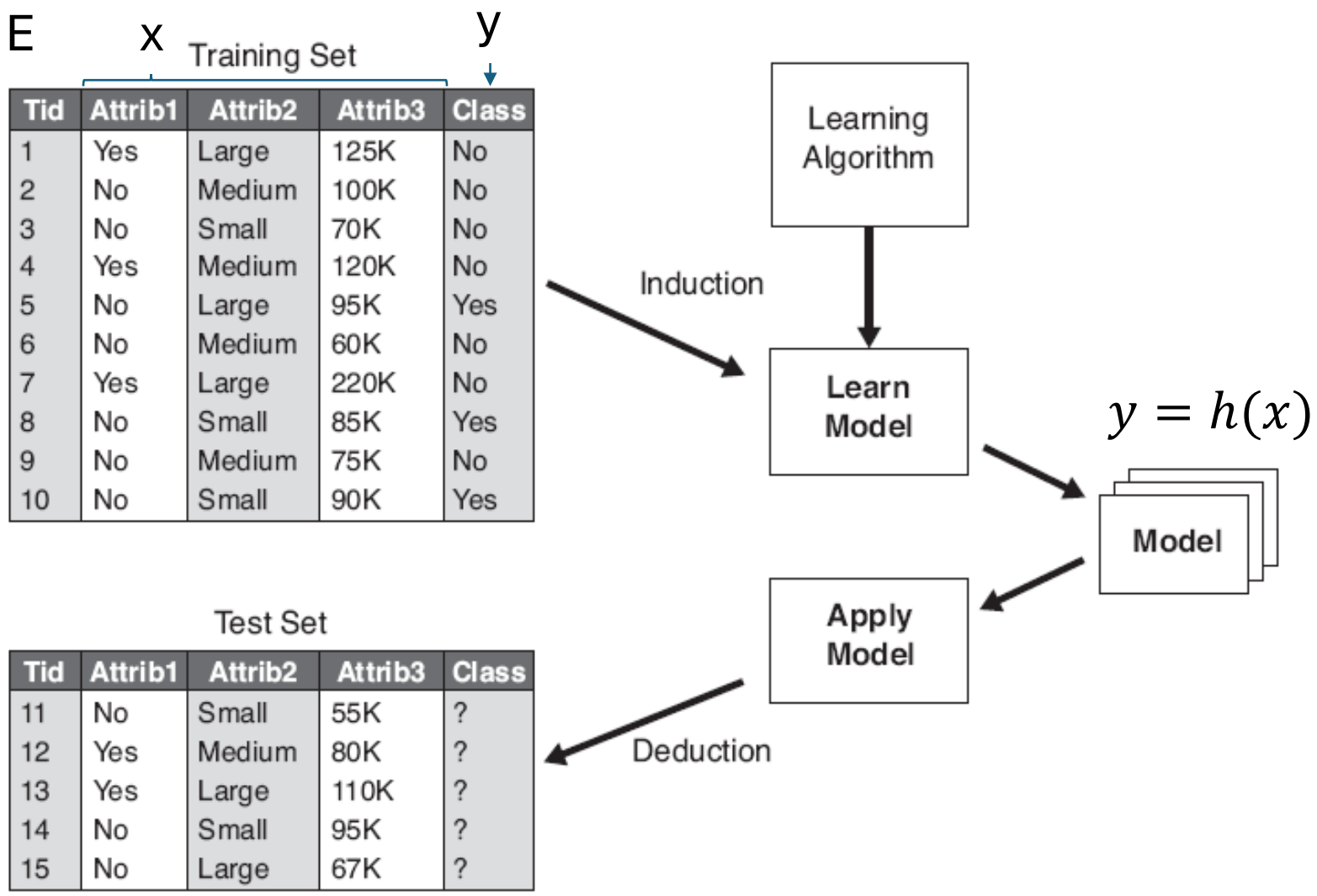
- Learning problem
  - —Given a hypothesis space H
  - —Find a hypothesis $h \in H$ such that $\hat{y}_i = h(x_i) \approx y_i$
  - —That is, we want to approximate f by h using E.



- Includes
  - —**Regression** (outputs = real numbers). Goal: Predict the number accurately. E.g., x is a house and $f(x)$ is its selling price.
  - —**Classification** (outputs = class labels). Goal: Assign new records to a class. E.g., x is an email and $f(x)$ is spam / ham

You already know linear regression. We focus on Classification.

Illustrating Classification Task

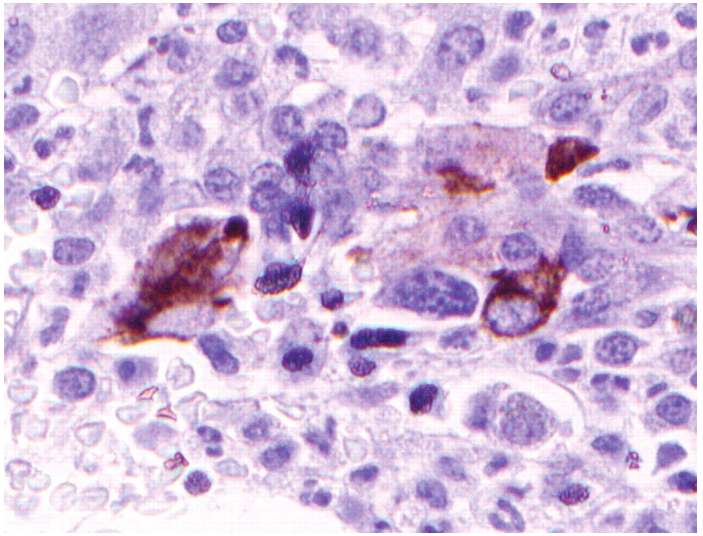


Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Figure 4.3. General approach for building a classification model.

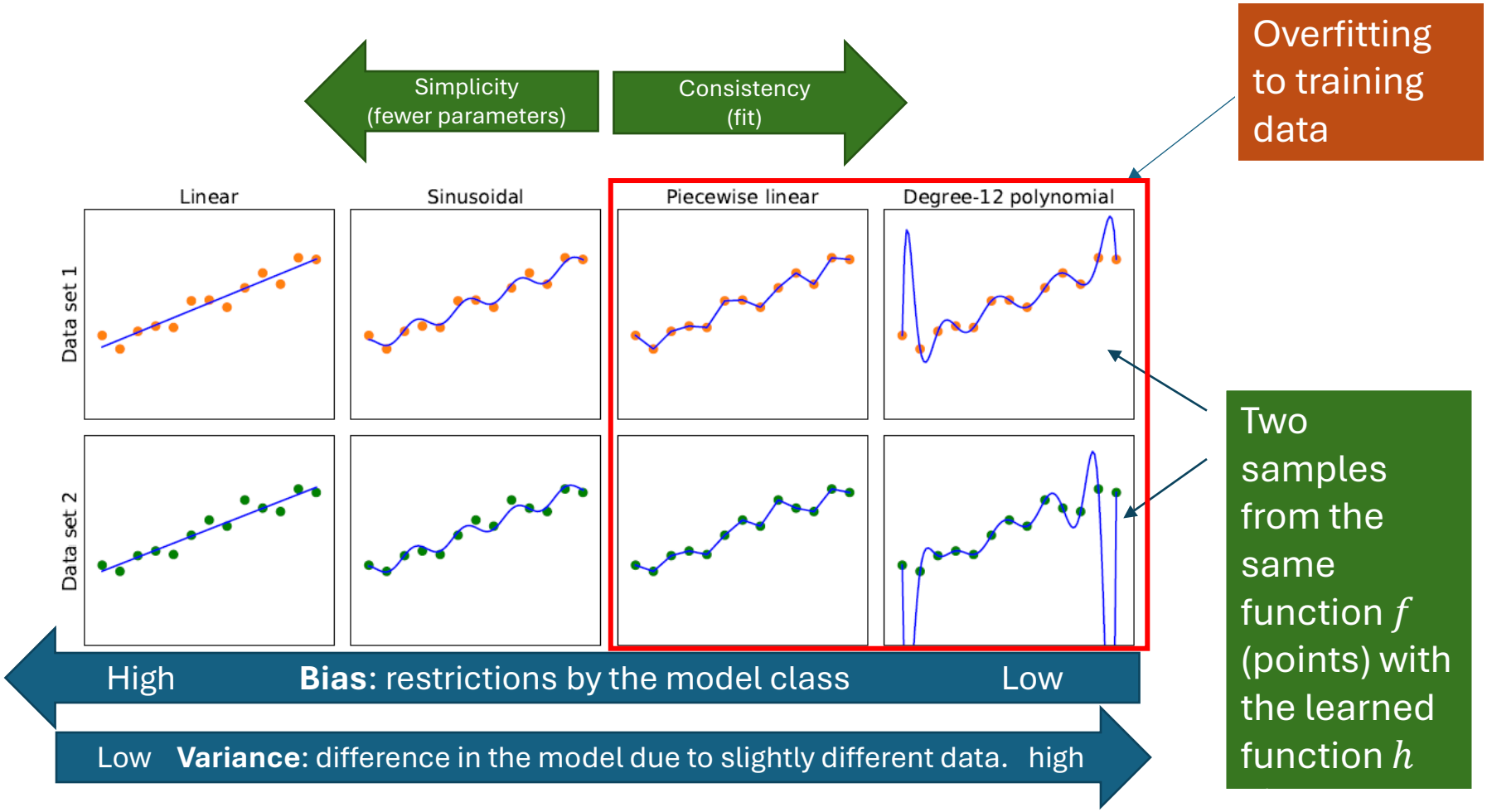
Examples of Classification Task



- Predicting tumor cells as benign or malignant.
- Classifying credit card transactions as legitimate or fraudulent.
- Categorizing news stories as finance, weather, entertainment, sports, etc.

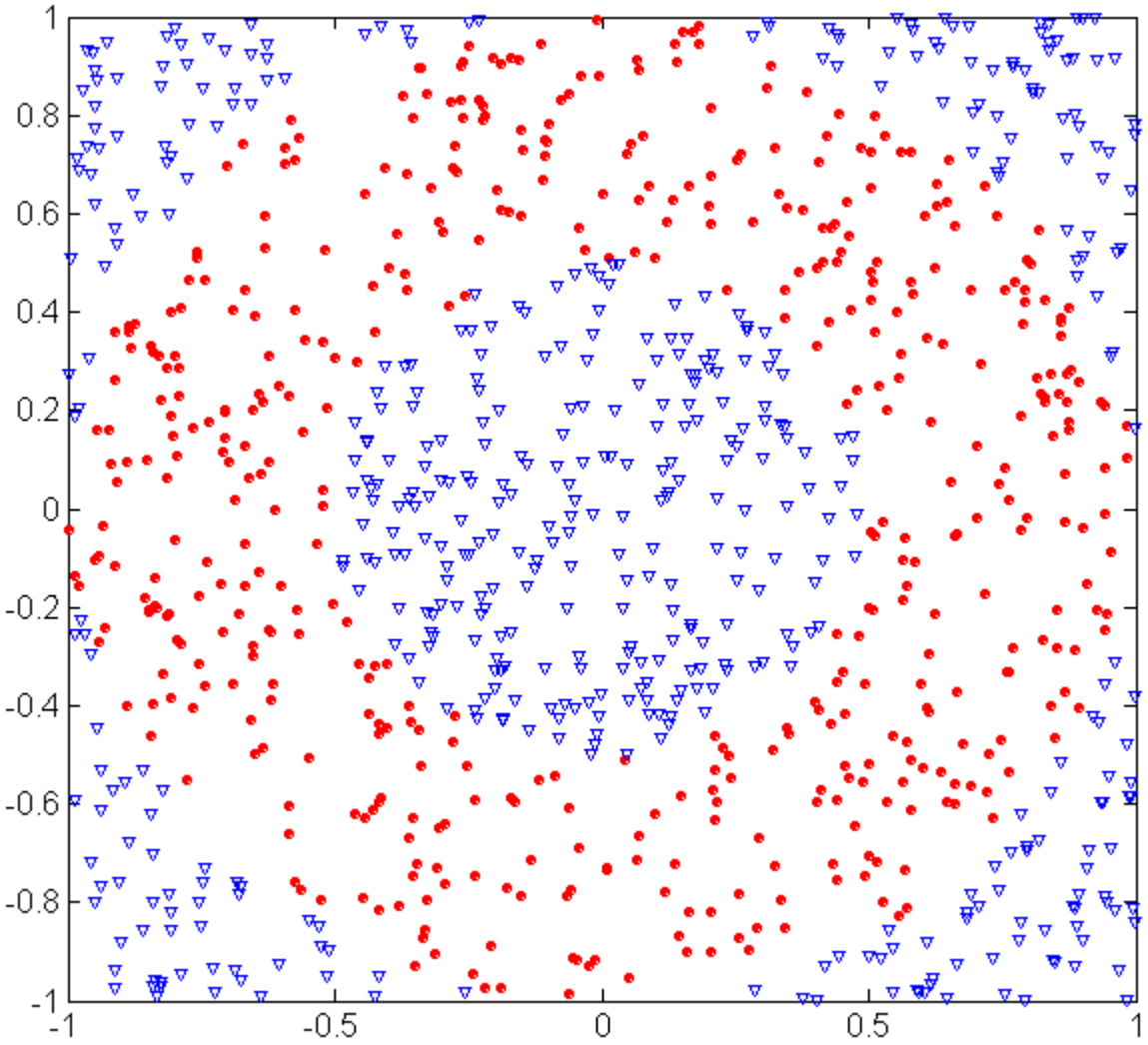


Model Selection: Bias vs. Variance



Note: This trade-off applies to any model.

Example: Underfitting and Overfitting



How is the data generated?

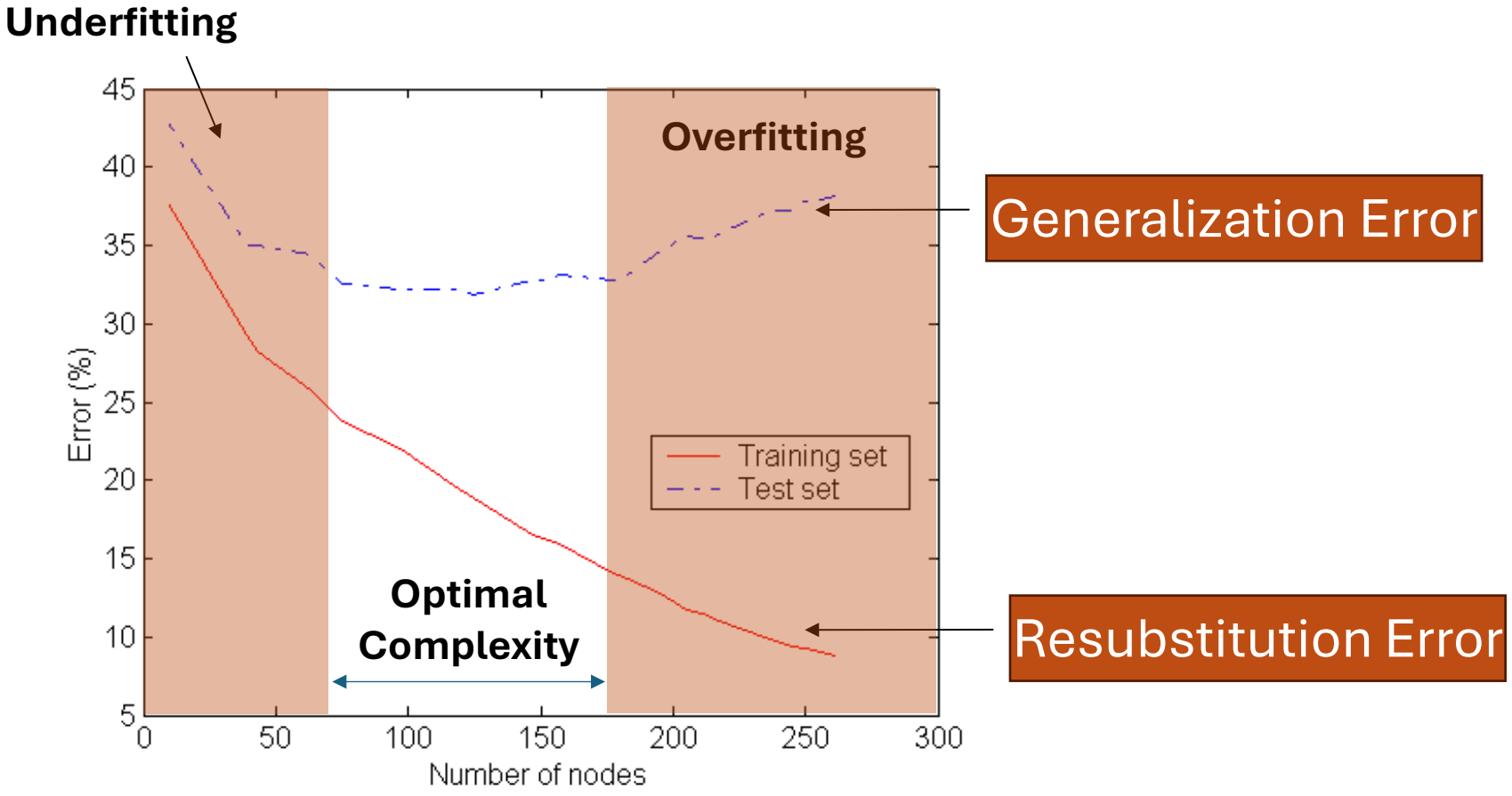
500 circular and 500 triangular data points.

Circular points:
$0.5 \geq \ sqrt({x_1}^2 + {x_2}^2) \leq 1$

Triangular points:

$sqrt({x_1}^2 + {x_2}^2) < \ 0.5$ or
$sqrt({x_1}^2 + {x_2}^2) \ > \ 1$

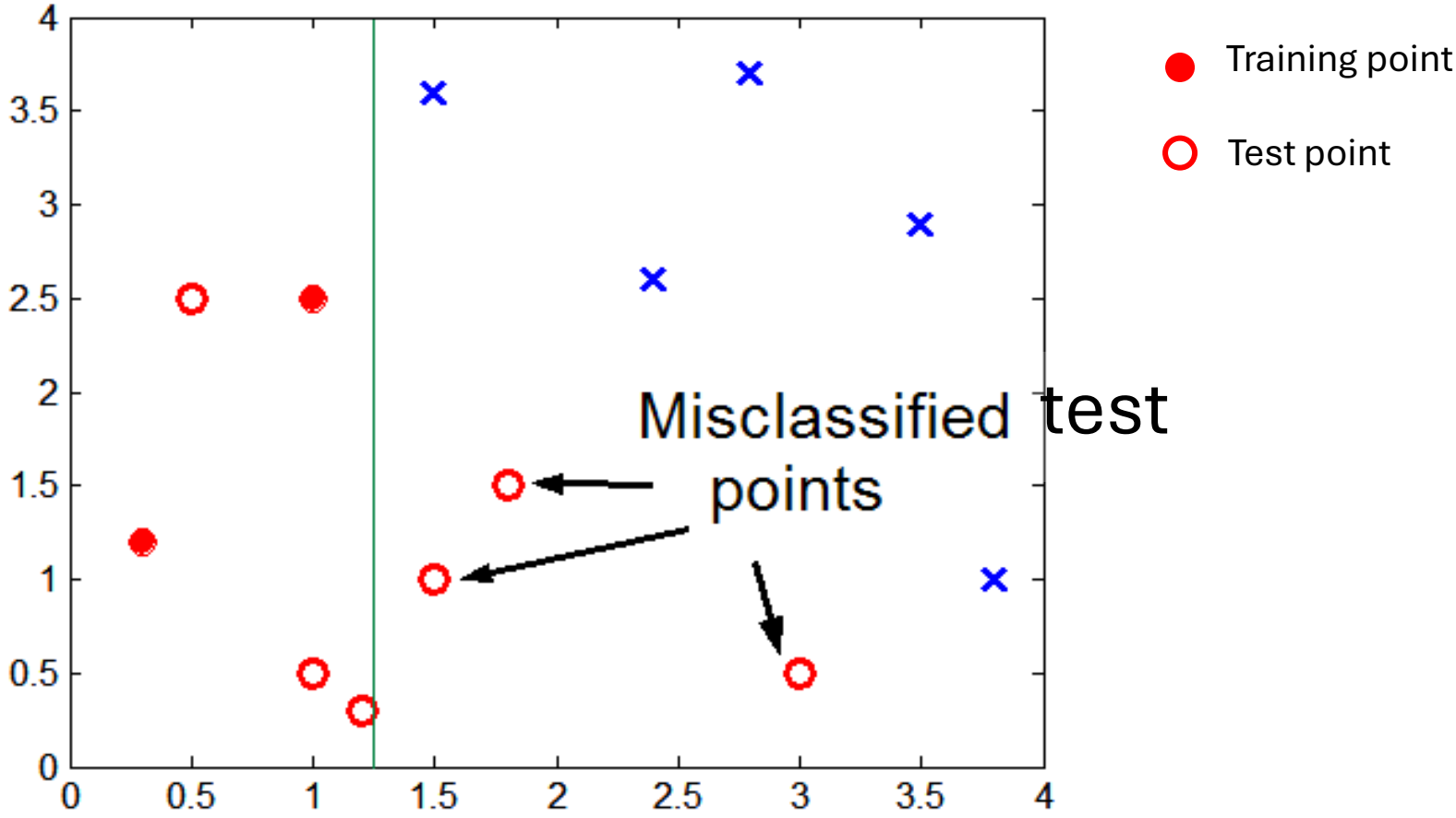
Example: Underfitting and Overfitting



Underfitting: The model is too simple, both training and test errors are large.

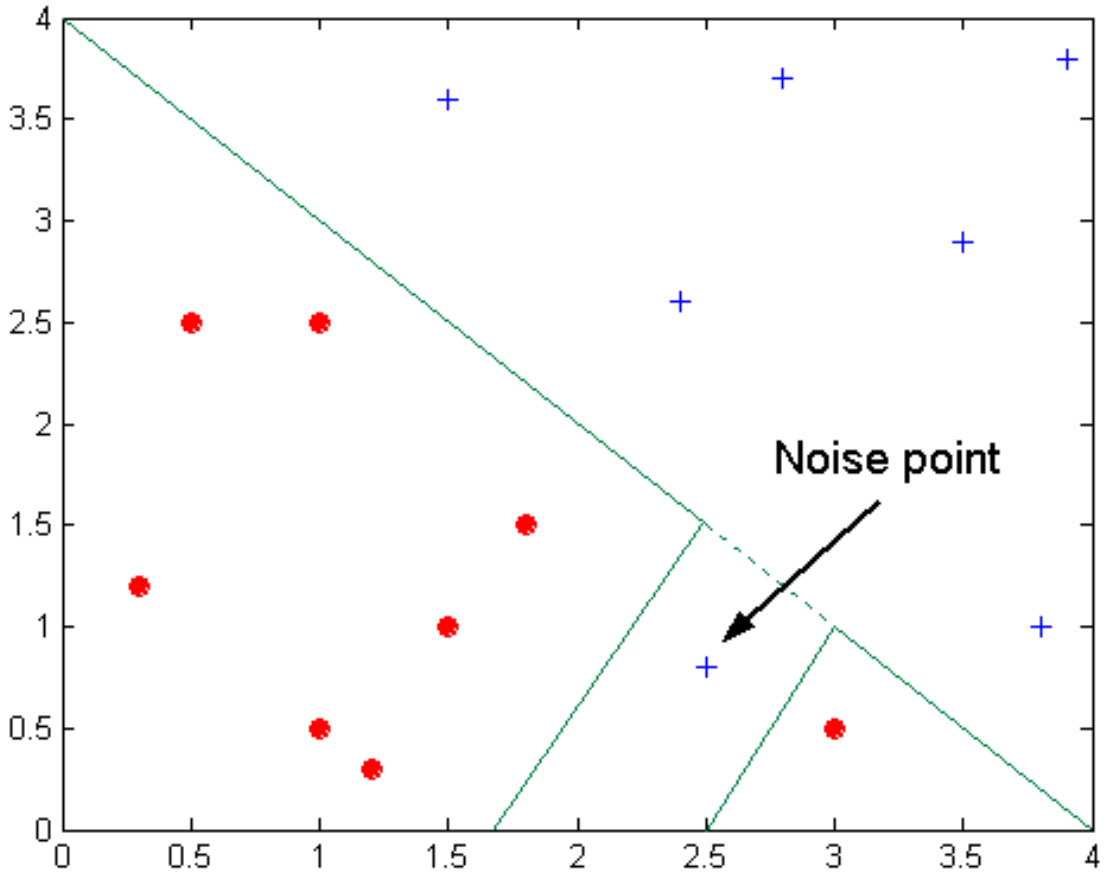
Overfitting: The model is too complicated and starts memorizing the training data. Generalization error goes up again.

Example: Underfitting due to Insufficient Examples



Lack of training data points in the lower half of the diagram makes it difficult to predict correctly the class labels of that region

Example: Overfitting due to Noise



Decision boundary is distorted to accommodate a noise point

Training Error vs. Generalization Error

- Training error is reduced by **overfitting** and results in decision trees that **are more complex than necessary.**

- Training error does not provide a good estimate of how well the tree will perform on new example (e.g., test data).

- We need to estimate the **Generalization Error** expected for new data.

Estimating the Generalization Error

- **Resubstitution error e**: error on training set
- **Generalization error e':** error on testing set

Methods for estimating generalization errors:

1. **Optimistic approach**: assume $e' = e$
2. **Pessimistic approach:**
   — Estimate as $e' = e + N \times 0.5$ (N: number of leaf nodes)
   — For a tree with 30 leaf nodes and 10 errors on training out of 1000 training instances:
   Training error $e = 10/1000 = 1\%$
   Estimated generalization error $e' = (10 + 30\ x\ 0.5)/1000 = 2.5\%$
3. **Validation approach:**
   — uses a validation (test) data set (or cross-validation) to estimate the generalization error.

Penalty for model complexity!
0.5 per leave node is often used for binary splits.



"Simpler is better"

Occam's Razor

-

The Principle of Parsimony

- Given two models of similar generalization errors, one should prefer the simpler model over the more complex model.
- Reason: Complex models have a greater chance of overfitting. I.e., it fitted accidentally errors in the training data.

Therefore, one should consider also model complexity when evaluating a model.



Topics

- Introduction
- Decision Trees
  - —Overview
  - —Tree Induction
- Overfitting and other Practical Issues
- **Model Selection and Evaluation**
  - —**Metrics for Performance Evaluation**
  - —Methods to Obtain Reliable Estimates
  - —Model Comparison (Relative Performance)
- Feature Selection

Metrics for Performance Evaluation: Confusion Matrix

- Focuses on the predictive capability of a model (not speed, scalability, etc.)
- For simplicity, we will present a binary classification problem here, but most measures generalize to multi-class problems.

Confusion Matrix

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	**a** **(TP)**	*b* *(FN)*
	Class=No	*c* *(FP)*	**d** **(TN)**

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Metrics for Performance Evaluation: Statistical Test

From Statistics: Null Hypotheses $H0$ is that the actual class is Yes.

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes		**Type I error** (FN)
	Class=No	**Type II error** (FP)	

← $H0$

Type I error: $P(\text{NO} \mid H0 \text{ is true})$ → Significance level α

Type II error: $P(\text{Yes} \mid H0 \text{ is false})$ → Power $1 - \beta$

Metrics for Performance Evaluation: Accuracy

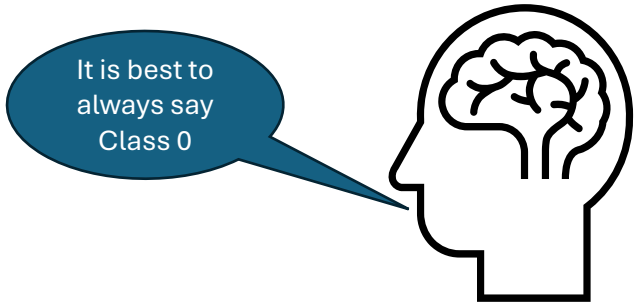
Most widely-used metric:

- How many do we predict correct (in percent)?

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

$$Accuracy = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{N}$$

Limitation of Accuracy



Consider a 2-class problem with a total population of

- Number of Class 0 examples = 9990
- Number of Class 1 examples = 10

A model that predicts everything to be class 0, has an accuracy of

$$9990/10000 = 99.9\ \%$$

Accuracy is misleading because the model does not detect any class 1 example!

→ This is a very common problem called the **class imbalance problem**

Cost Matrix

Different types of error can have different cost!

		PREDICTED CLASS	
ACTUAL CLASS	C(i|j)	**Class=Yes**	**Class=No**
	Class=Yes	C(Yes|Yes)	C(No|Yes)
	Class=No	C(Yes|No)	C(No|No)

$C(i \mid j)$: Cost of misclassifying class j example as class i

Computing the Cost of Classification

Cost Matrix	PREDICTED CLASS		
	C(i|j)	+	-
ACTUAL CLASS	+	-1	100
	-	1	0

Missing a '+' case is really expensive!

Model M_1	PREDICTED CLASS		
		+	-
ACTUAL CLASS	+	150	40
	-	60	250

Accuracy = 80%

Cost = -1*150+100*40+
1*60+0*250 = **3910**

Model M_2	PREDICTED CLASS		
		+	-
ACTUAL CLASS	+	250	45
	-	5	200

Accuracy = 90%

Cost = **4255**

Cost-Biased Measures (from Information Retrieval)

		PREDICTED CLASS	
		Class Yes	Class No
ACTUAL CLASS	Class Yes	a (TP)	b (FN)
	Class No	c (FP)	d (TN)

$$Precision\ (p) = \frac{a}{a + c}$$

$$Recall\ (r) = \frac{a}{a + b}$$

$$F - measure\ (F) = \frac{2rp}{r + p} = \frac{2a}{2a + b + c}$$

- Precision only considers cost for examples predicted as Yes.
- Recall only considers cost for examples that are truly Yes.
- F-measure combines precision and recall and ignores d.

Kappa Statistic

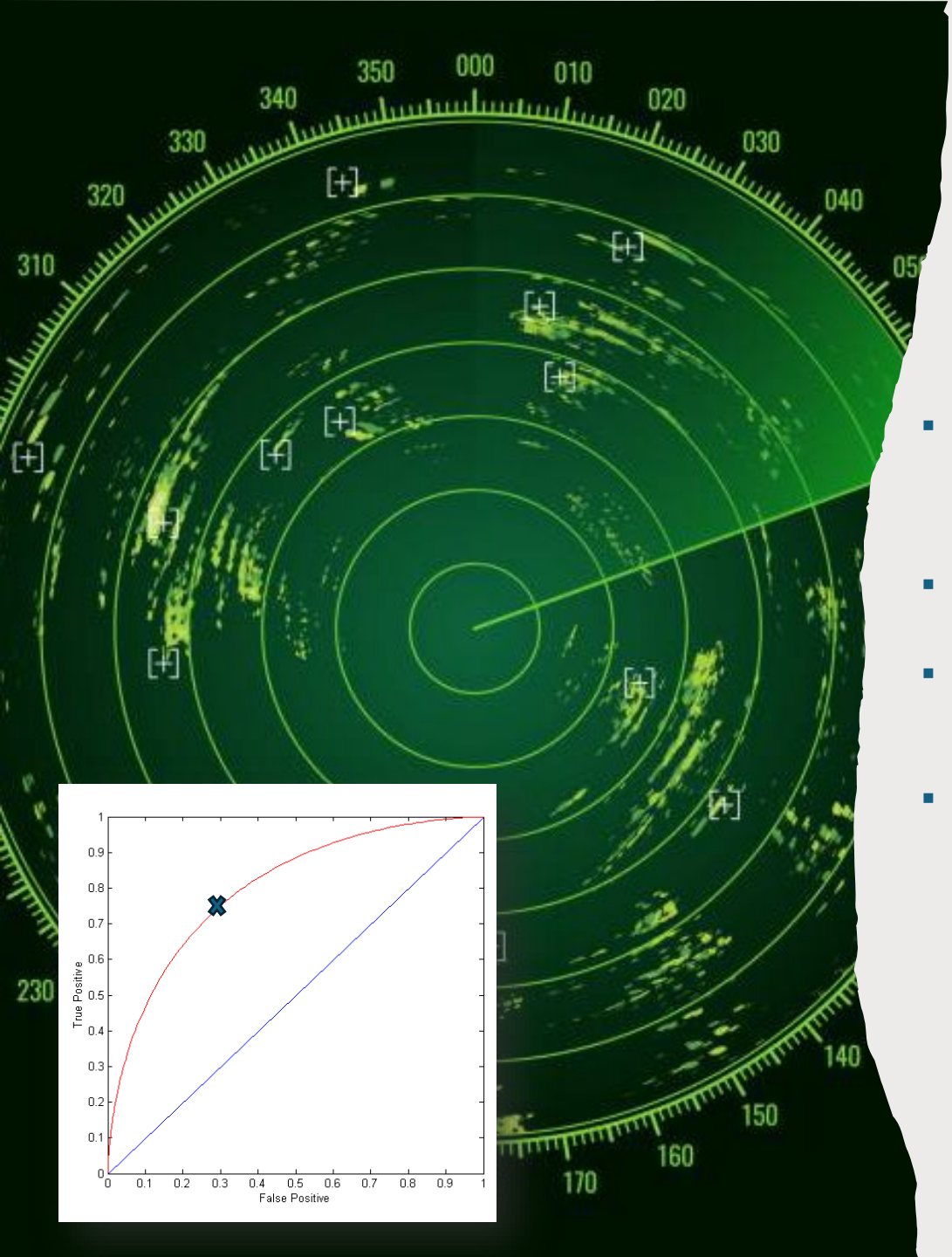
Idea: Compare the accuracy of the classifier with a **random classifier**. The classifier should be better than random!

		PREDICTED CLASS	
		Class Yes	Class No
ACTUAL CLASS	Class Yes	a (TP)	b (FN)
	Class No	c (FP)	d (TN)

$$\kappa = \frac{\text{total accuracy} - \text{random accuracy}}{1 - \text{random accuracy}}$$

$$\text{total accuracy} = \frac{TP + TN}{N}$$

$$\text{random accuracy} = \frac{TP + FP \times TN + FN + FN + TN \times FP + TP}{N^2}$$

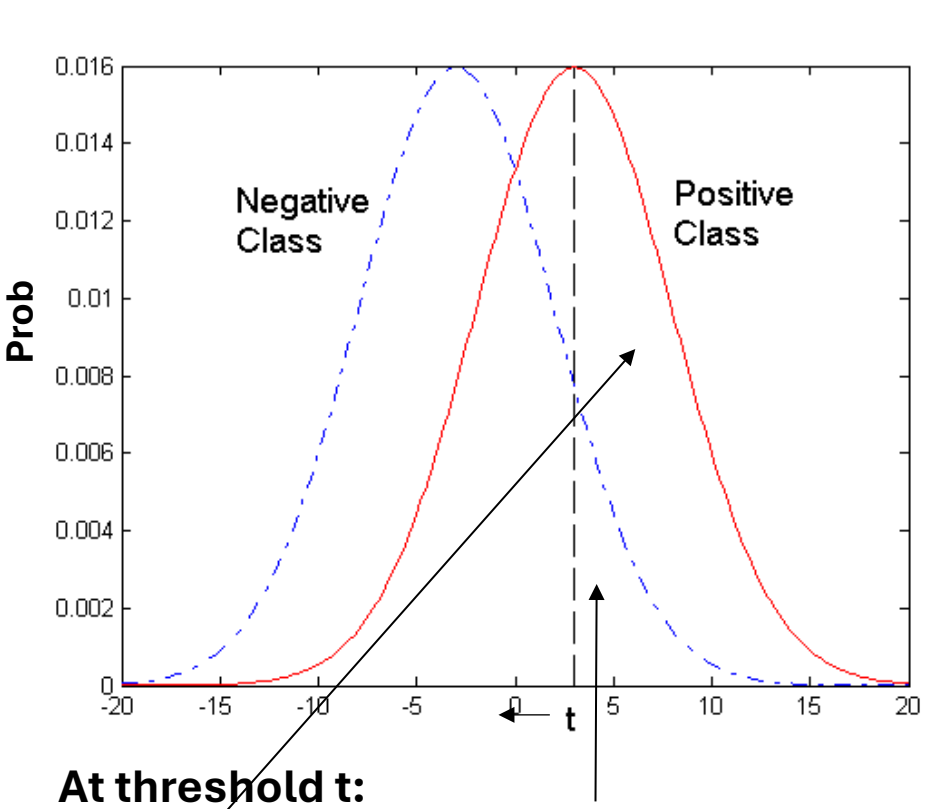


Receiver Operating Characteristic (ROC)

- Developed in 1950s for signal detection theory to analyze noisy signals to characterize the trade-off between positive hits and false alarms.
- Works only for binary classification (two-class problems).
- ROC curve plots TPR (true positive rate) on the y-axis against FPR (false positive rate) on the x-axis.
- Performance of each classifier represented as a point. Changing the threshold of the algorithm, sample distribution or cost matrix changes the location of the point and forms a curve.

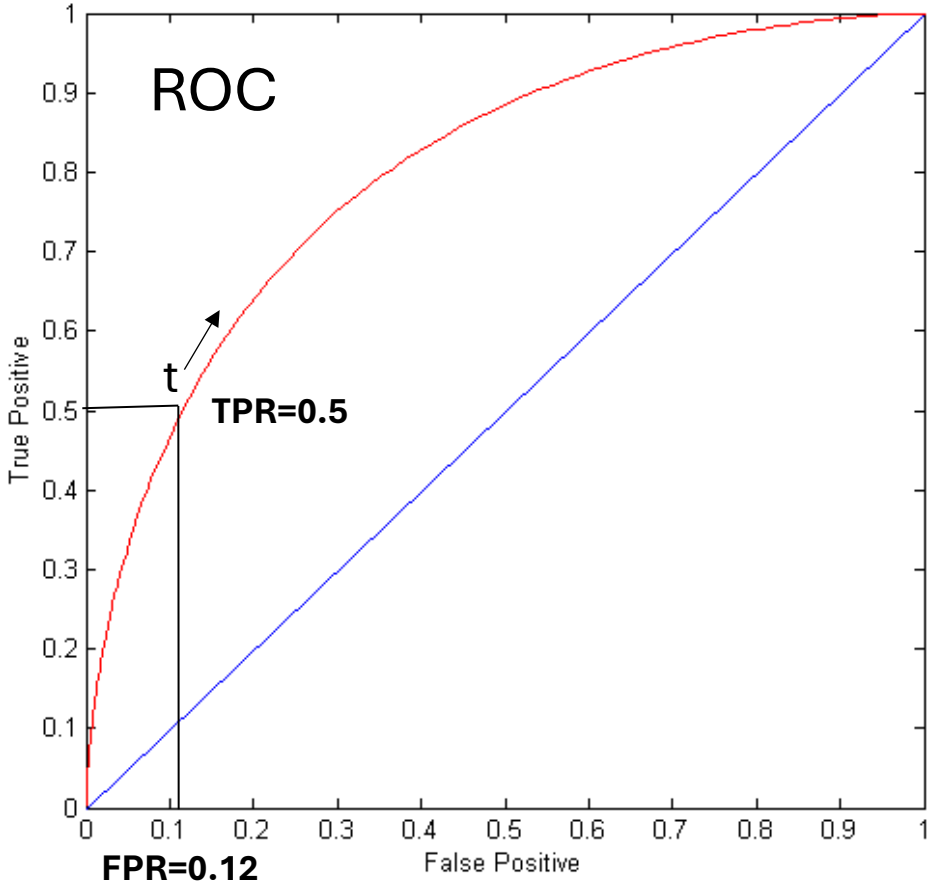
ROC Curve

- Example with 1-dimensional data set containing 2 classes (positive and negative)
- Any points located at x > t is classified as positive



At threshold t:

TPR=0.5, FNR=0.5, **FPR=0.12,** FNR=0.88



- Move t to get the other points on the ROC curve.

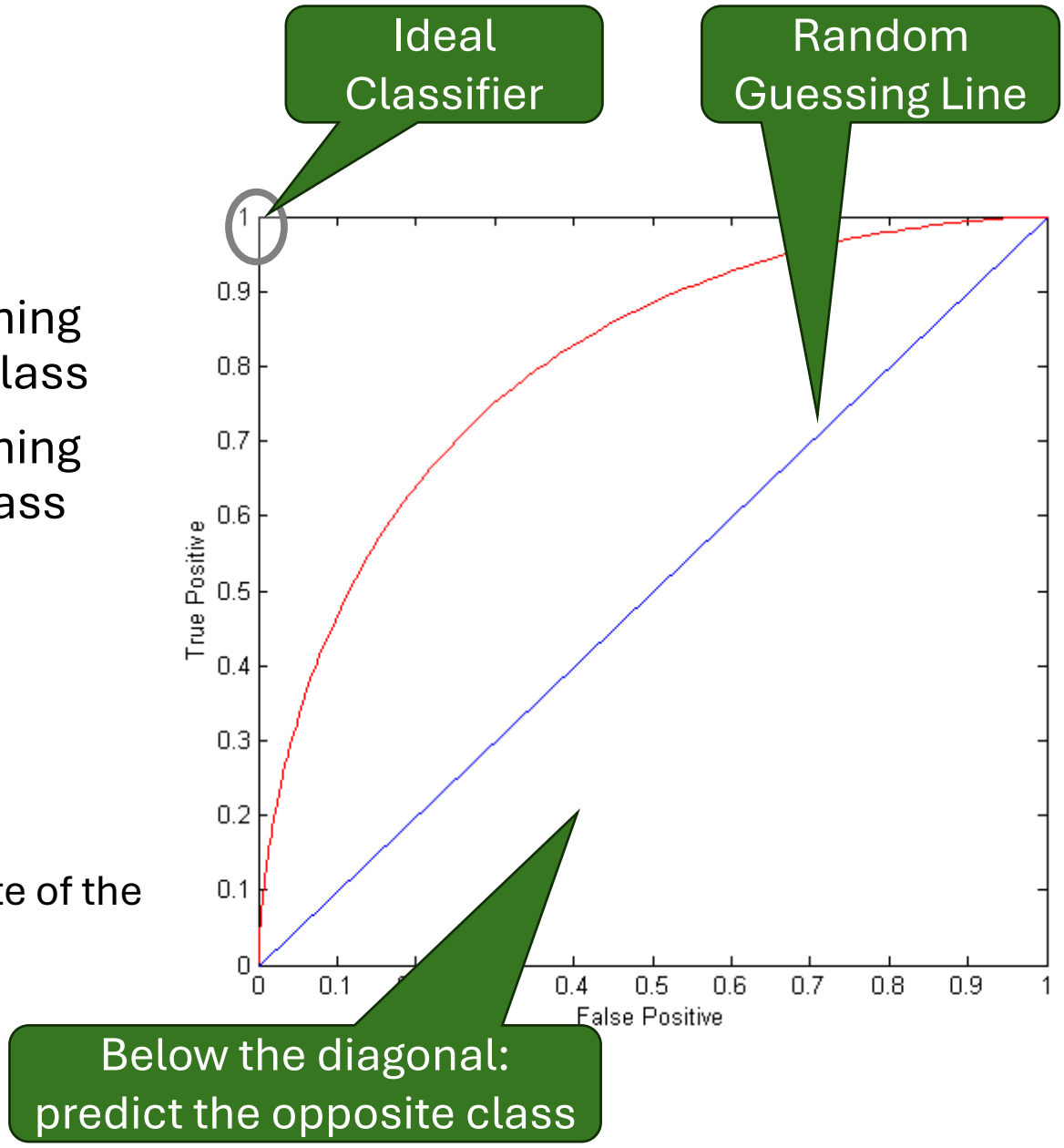
ROC Curve

(TPR,FPR):

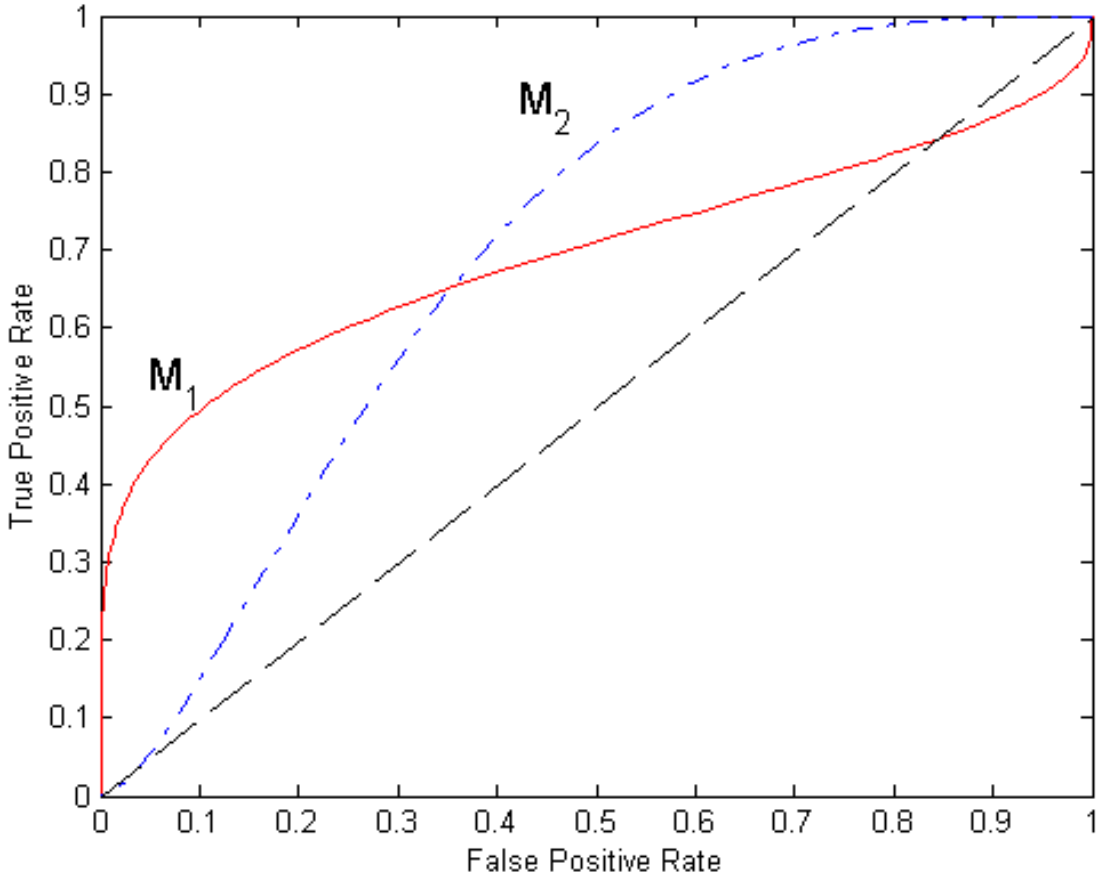
- (0,0): declare everything to be negative class
- (1,1): declare everything to be positive class
- (1,0): ideal

Diagonal line:

- Random guessing
- Below diagonal line: prediction is opposite of the true class



Using ROC for Model Comparison



No model consistently outperform the other

- M1 is better for small FPR
- M2 is better for large FPR

Area Under the ROC curve (AUC)

- Ideal:
  - AUC = 1
- Random guess:
  - AUC = 0.5

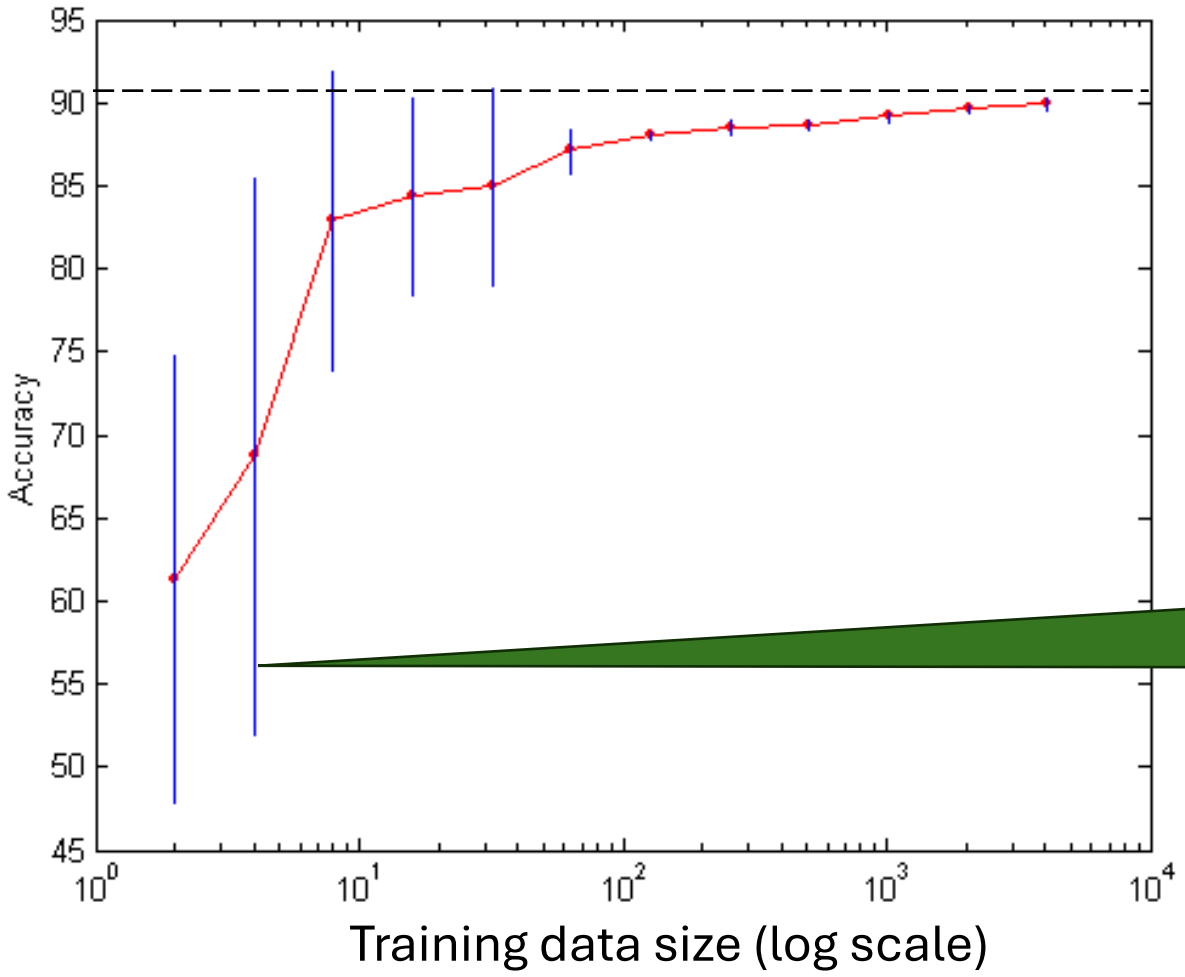


Topics

- Introduction
- Decision Trees
  - —Overview
  - —Tree Induction
- Overfitting and other Practical Issues
- Model Selection and Evaluation
  - —Metrics for Performance Evaluation
  - —**Methods to Obtain Reliable Estimates**
  - —Model Comparison (Relative Performance)
- Feature Selection

Learning Curve

Accuracy and variance between runs depend on the size of the training data.



Learning curve shows how accuracy on unseen examples changes with varying training sample size

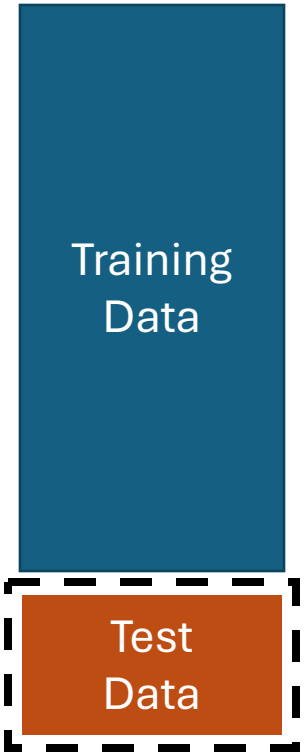
Variation for different runs

Estimating the Generalization Error Using Test Data

- To estimate generalization error we need to separate the data into a set to train and a set to test.

- **Holdout testing/Random splits**: Split the data randomly into, e.g., 80% training and 20% testing.

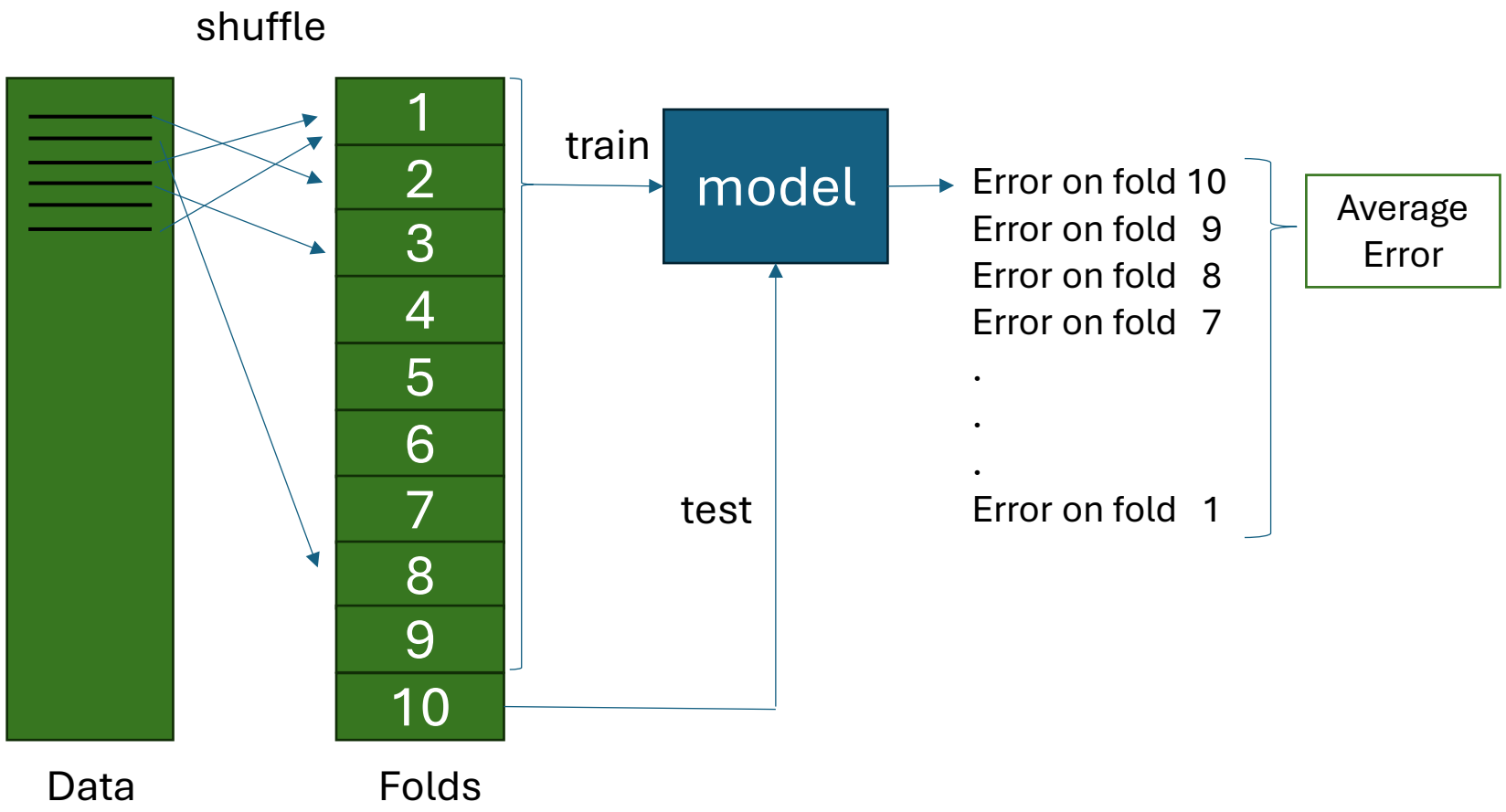
Very important: the algorithm can never look at the test set during learning!



k-fold Cross Validation

k-fold cross validation: Use data better to estimate the generalization error:

- Split the data randomly into k folds.
- For k rounds hold 1 fold back for testing and use the remaining $k - 1$ folds for training.
- Use the average of the error/accuracy as a better estimate.
- Some algorithms/tools do that internally.



Training and Testing with Hyperparameters

Hyperparameters: Many algorithms allow choices for learning. E.g.,

- —maximal decision tree depth
- —selected features

We do not want to overfit the hyperparameters!!!

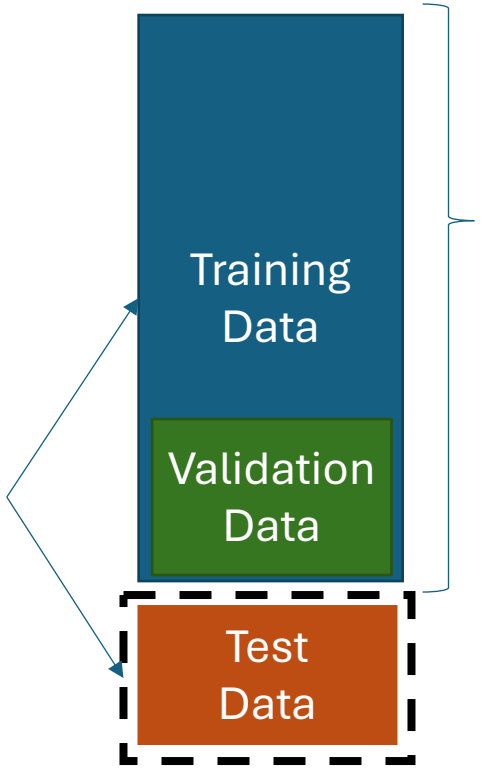
Use a generalization error estimate twice:

1. **Train:** Learn models on the **training data** (without the validation data) using different hyperparameters.
   - —A grid of possible hyperparameter combinations
   - —greedy search
2. **Model Selection:** Evaluate the models using the **validation data** and choose the hyperparameters with the best accuracy. Rebuild the model using all the training data.
3. **Test** the final model using the **test data**.



Typical Data Use with Model Selection

Test data: Split the data randomly into 20% testing and 80% training + validation.



Model Selection: Use training & validation data with 10-fold cross validation for choosing between models and hyper parameter tuning.

Confidence Interval for Accuracy

- The observed accuracy is an **estimate** of the true accuracy of the model. How good is the estimate?

- Each prediction can be regarded as a **Bernoulli trial**: A Bernoulli trial (a biased coin toss) has 2 possible outcomes:
  heads (correct) or tails (wrong)

  We use p for the true chance that a prediction is correct (= true accuracy).



- Predictions for a test set of size N are a collection of N Bernoulli trials. The number of correct predictions x has a **Binomial distribution**:
  $$X \sim Binomial(N, p)$$

- Example: Toss a fair coin 50 times, how many heads would turn up?
  Expected number of heads $E[X] = Np = 50 \times 0.5 = 25$

- **Application for Accuracy**: If we observe x correct predictions then the observed accuracy is
  $$\hat{p} = x/N$$

Can we give bounds for the true accuracy of model p?

Confidence Interval for Accuracy

For large test sets ($N > 30$) we can approximate the Binomial distribution

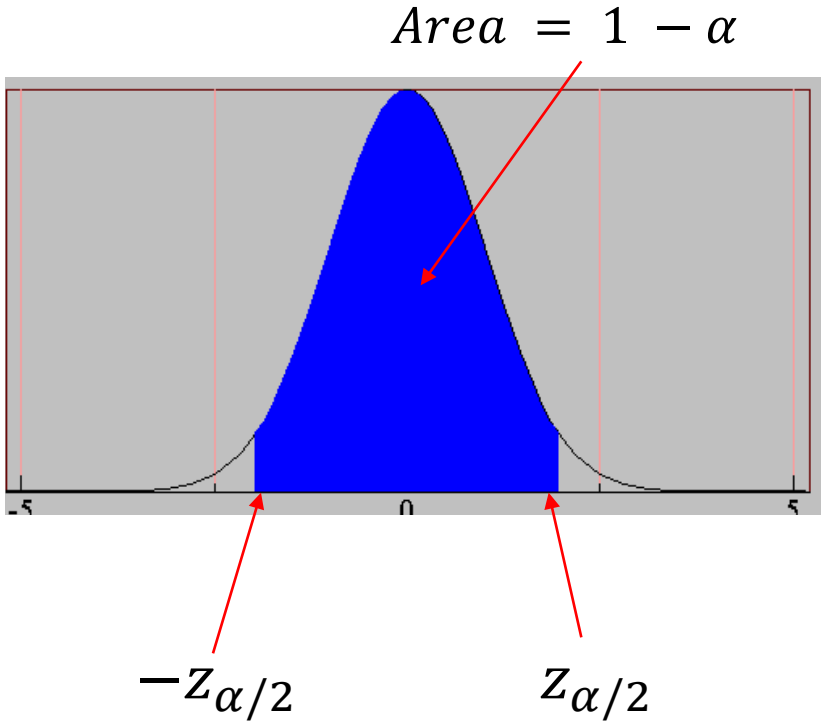
$$X \sim Binomial(N, p)$$

by a Normal distribution:

$$X \sim Normal(Np, Np(1-p))$$

Confidence Interval for $p = \frac{X}{N}$ (Wald Method):

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{N}}$$



Confidence Interval for Accuracy

Consider a model that produces an accuracy of 80% when evaluated on 100 test instances:

1. $N = 100,\ acc = 0.8$
2. Let $1 - \alpha = 0.95$ (95% confidence)
3. Find the critical value for the normal distribution. $z_{\alpha/2} = 1.96$
4. Calculate the interval around the accuracy.

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{N}} = \begin{cases} 0.722 \\ 0.878 \end{cases}$$

$1 - \alpha/2$	$z_{\alpha/2}$
0.99	2.58
0.98	2.33
0.95	1.96
0.90	1.65

Table or
R $qnorm(1 - \alpha/2)$

Data mining tools typically calculate this for us.



Topics

- Introduction
- Decision Trees
  - —Overview
  - —Tree Induction
- Overfitting and other Practical Issues
- Model Selection and Evaluation
  - —Metrics for Performance Evaluation
  - —Methods to Obtain Reliable Estimates
  - —**Model Comparison (Relative Performance)**
- Feature Selection

Comparing Performance between 2 Models

Given two models, say M_1 and $M2$, which is better? This is a statistical **model selection** problem.

For large test sets ($N > 30$) we can approximate the observed accuracies (sampled from a Binomial distribution) using the true but unknown model accuracies p_1 and p_2:

$$acc_1 \sim Normal(Np_1, Np_1(1-p_1))$$
$$acc_2 \sim Normal(Np_2, Np_2(1-p_2))$$

Perform a paired t-test with:

H0: There is no difference between the observed accuracies of the models.

H1: There is a difference.

Notes

- **Hyperparameter** tuning is also a model selection problem.
- Comparing more than two models: You need to **correct for multiple comparisons!** For example, using Bonferroni correction or False Discovery Rate (FDR).



Topics

- Introduction
- Decision Trees
  - —Overview
  - —Tree Induction
- Overfitting and other Practical Issues
- Model Selection and Evaluation
  - —Metrics for Performance Evaluation
  - —Methods to Obtain Reliable Estimates
  - —Model Comparison (Relative Performance)
- **Feature Selection**

Feature Selection

What features should be used in the model?

Univariate feature importance score

- Measures how related each feature is to the class variable.
- E.g., chi-squared statistic, information gain.

Feature subset selection

- Tries to find the best set of features.
- Often uses a black box approach where different subsets are evaluated using a greedy search strategy.
- E.g.: Stepwise backward selection tries to remove one feature at a time.



Conclusion

- Classification is **supervised learning** with the goal to find a model that predicts well (i.e., has a low generalization error).
- **Generalization error** can be estimated using test sets/cross-validation and should be used for model selection.
- Model evaluation and comparison needs to take **model complexity** into account.